[1]:
```
import pandas as pd
```

[2]:
```
registrations = pd.DataFrame({'reg_id':[1,2,3,4],'name':['Andrew','Bobo','Claire','David']})
logins = pd.DataFrame({'log_id':[1,2,3,4],'name':['Xavier','Andrew','Yolanda','Bobo']})
```

[3]:
```
registrations
```

[3]:

	reg_id	name
0	1	Andrew
1	2	Bobo
2	3	Claire
3	4	David

[4]:
```
logins
```

[4]:

	log_id	name
0	1	Xavier
1	2	Andrew
2	3	Yolanda
3	4	Bobo

[6]:
```
help(pd.merge)
```

```
Help on function merge in module pandas:

merge(
    left: DataFrame | Series,
    right: DataFrame | Series,
    how: MergeHow = 'inner',
    on: IndexLabel | AnyArrayLike | None = None,
    left_on: IndexLabel | AnyArrayLike | None = None,
    right_on: IndexLabel | AnyArrayLike | None = None,
    left_index: bool = False,
    right_index: bool = False,
    sort: bool = False,
    suffixes: Suffixes = ('_x', '_y'),
    copy: bool | lib.NoDefault = <no_default>,
    indicator: str | bool = False,
    validate: str | None = None
) -> DataFrame
    Merge DataFrame or named Series objects with a database-style join.
```

[7]:
```
pd.merge(registrations,logins,how='inner',on='name')
```

[7]:

	reg_id	name	log_id
0	1	Andrew	2
1	2	Bobo	4

So, this is like, intersection of two datasets

[8]:
```
pd.merge(logins,registrations,how='inner',on='name')
```

[8]:

	log_id	name	reg_id
0	2	Andrew	1
1	4	Bobo	2

In real life, its order doesn't matter much.

"Inner" returns the values that are present in both the tables (here=both) -- intersection ¶